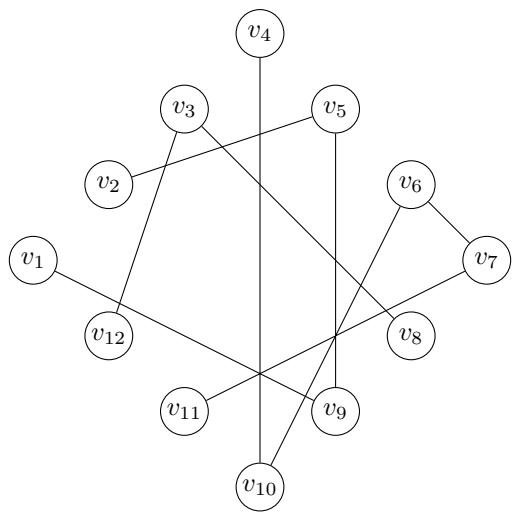
Definitions and introduction

We want a data structure that can be used to effectively represent disjoint sets. That is, each element is in exactly one set. The abstract data type that we will develop shall allow the following operations:

`MakeSet(`x`)`	Create a new set containing element x
`Union(`x`,`y`)`	Union the sets containing x and y
`InSameSet(`x`,`y`)`	Returns true if x and y are in the same set
`FindSet(`x`)`	Returns a reference to a representative element of the set containing x

Note that `Intersection`(x, y) is not an operation that we must considere since the sets are disjoint and therefore any intersection is empty. The benefits and applications for this data structure may not be as immediately obvious to some readers as some of the previous data structures. The first application we shall note is that this abstract data type can perfectly handle the disjoint sets that were present during Kurskal's Minimum Spanning Tree Algorithm. Our second application is the following:
Given a graph G, what are the connected components[1]?



How can we tell if v_i and v_j are in the same connected component? We can create disjoint sets representing the connect components of the graph and then query the sets to determine if two vertices are in the same connected component. Pseudocode for solving this problem is given on the following page.

[1]The connected components of a graph G are the disjoint subsets of the graph such that each subset is connected.

We first build the sets corresponding to the connected components

```
foreach vertex v_i ∈ G.V do
    MakeSet(v_i)
end for
foreach edge (v_i, v_j) ∈ G.E do
    Union(v_i,v_j)
end for
```

Once the sets are constructed we can then implement the following method for determining if two vertices are connected.

```
function AreConnected(v_i,v_j)
    if InSameSet(v_i,v_j) then
        return true
    else
        return false
    end if
end function
```

We note that we can easily determine if two elements are in the same set by using the `FindSet` function. Since `FindSet` returns a representative element from the set, if two elements have the same representative then they are in the same set.

```
function InSameSet(x,y)
    if FindSet(x) = FindSet(y) then
        return true
    else
        return false
    end if
end function
```

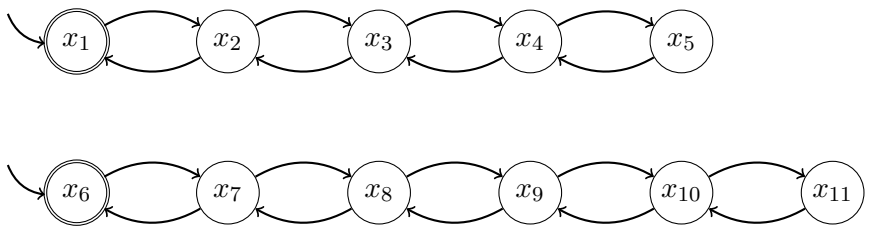
Hopefully the reader is convinced that this abstract data type has some applications – the question then becomes how can implement this efficiently? We shall introduce and discuss two implementations of the Union-Find data structure.

Linked List Union-Find

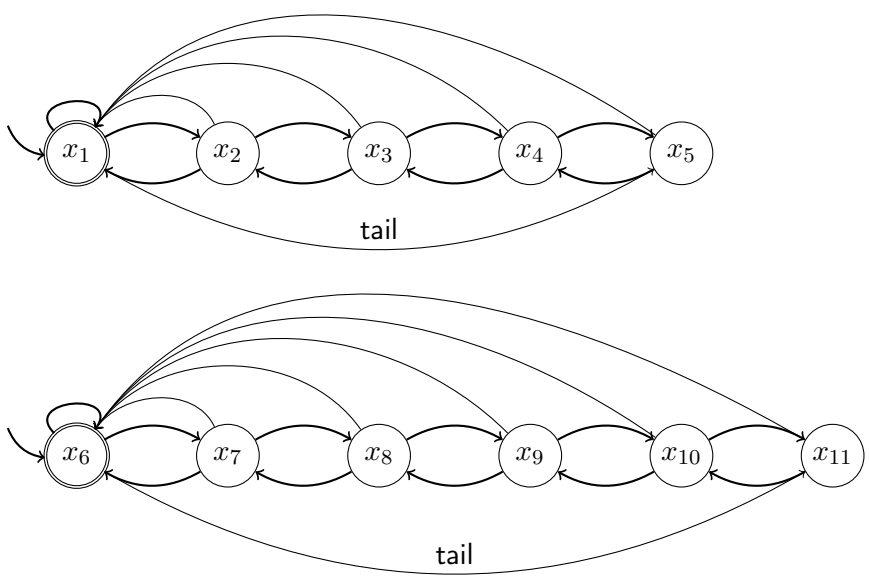
The first implementation of the Union-Find data structure that we shall discuss is the Linked List implementation. In this implementation each disjoint set is represented by a linked list and the representative element from each linked list will be the head of the linked list. For example, the disjoint sets

$$S_1 = \{x_1, x_2, x_3, x_4, x_5\} \qquad S_2 = \{x_6, x_7, x_8, x_9, x_{10}, x_{11}\}$$

with x_1 as the representative element from S_1 and x_6 as the representative element from S_2 would be represented as



We note that the linked lists as drawn above do not easily support `FindSet` since (in general) we have to traverse to entire list to find the head of the linked list. As anyone who has studied linked lists will have guessed, the solution here is to include a `head` pointer at each of the nodes of the linked list that indicates the head of the list. Similarly, we shall also include a `tail` pointer from the head to the tail of the list. Note that this will allow all nodes to access to tail in constant time.



With these final modifications, we can now discuss the implementation of the methods of the data structure.

```
function MakeSet(x)
    x.head = x
    x.tail = x
    x.next = NULL
end function
```

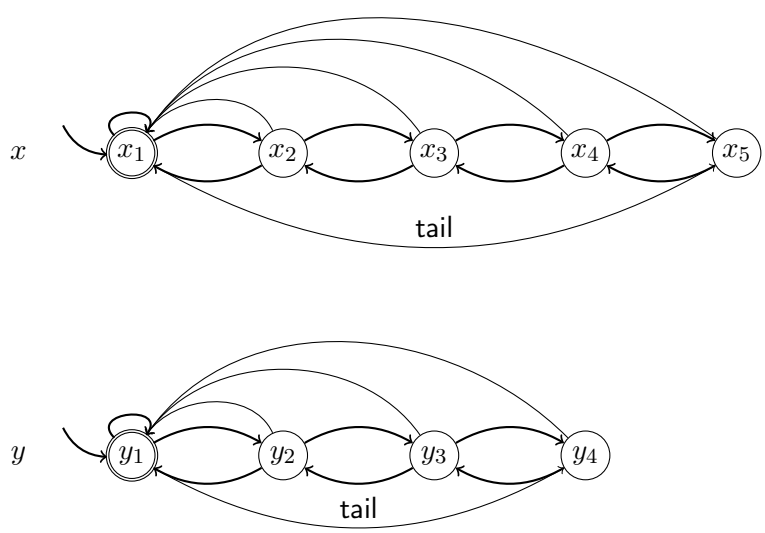
The running time of this method is clearly $\Theta(1)$.

```
function FindSet(x)
    return x.head
end function
```

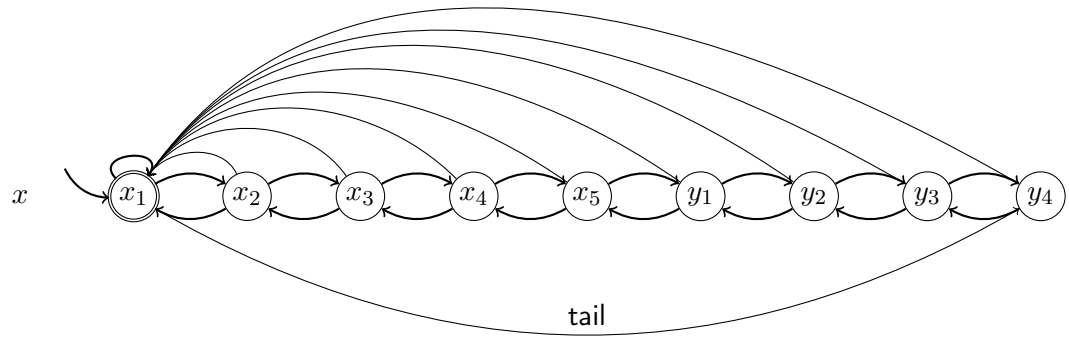
The running time of this method is clearly $\Theta(1)$. Our implementation of `Union` shall concatenate the two linked lists.

```
function Union(x,y)
    x' = FindSet(x)
    y' = FindSet(y)
    x'.tail.next = y'
    z = y'
    while z ≠ NULL do
        z.head = x
        z = z.next
    end while
    x'.tail = y'.tail
end function
```

The running time of this method will be $\Theta(y.\texttt{length})$ since the while loop iterates over every node in y. Consider the following disjoints sets



The result of $\texttt{Union}(x, y)$ will be



We note that there is an "obvious" way that we can make this method more efficient. The method as written will always take $\Theta(y.\texttt{length})$ since it always appends the nodes of y to the tail of x. What if instead of always iterating over the linked list y, we iterate over whichever linked list is shorter? Since the order of elements in set is irrelevant this should cause no issues!

```
function WeightedUnion(x,y)
    x' = FindSet(x)
    y' = FindSet(y)
    if x' = y' then
        return
    end if
    if x'.length ≥ y'.length then
        x'.tail.next = y'
        z = y'
        while z ≠ NULL do
            z.head = x
            z = z.next
        end while
        x'.length = x'.length + y'.length
        x'.tail = y'.tail
    else
        WeightedUnion(y,x)
    end if
end function
```

With this modification the running time now becomes $\Theta(\min\{x.\texttt{length}, y.\texttt{length}\})$ which is a guaranteed improvement. Note that this improvement does not come "free of cost" since we must now store the length of each linked list.

Our current running time for `WeightedUnion` is correct, but does not help us easily determine the running time of algorithms that call this method. Namely, it remains difficult to analyze the running time of our connected components problem we discussed at the start. To reconcile this issue we shall analyze the **Amortized** Worst Case running time. We shall analyze the running time of combining n initially disjoint elements into a single disjoint set. To begin with let x_i be any element in our data set. We shall track the number of times x_i.`head` gets updated.

Minimum size of set containing x_i	Cost of changing x_i.`head`
1	c
2	c
4	c
8	c
$\vdots$	$\vdots$
$n/2$	c
n	c

Note that for x_i.`head` to be updated, the set containing x_i must be unioned with a set that is **at least** as large as the set containing x_i. Therefore, the size of the set containing x_i must always at least double in size whenever x_i.`head` is updated. Let K_i be the number of times x_i.`head` gets updated. From the table presented above we can see that the maximum number of times that x_i.`head` could be updated is $\log_2(n)$, therefore,

$$K_i \leq \log_2(n)$$

Now, we note that the *cost* of updating x_i.`head` is constant and that the running time of building a single disjoint set of n elements is

$$T(n) = \text{Cost of updating } x_i\text{.head for } \textbf{all} \text{ nodes} \leq \sum_{i=1}^{n} cK_i = \sum_{i=1}^{n} c\log_2(n) = cn\log_2(n)$$

To achieve a lower bound on the worst case for combining all sets, we shall demonstrate that a *particular* set of `WeightedUnion` operations takes $\Theta(n\log_2(n))$ time and therefore the worst case must take at least as long. Consider the following scheme

1. Create n disjoint sets each containing a single of the n elements.
2. Pairwise union these sets such that we have $n/2$ sets of size 2.
3. Pairwise union the sets of size 2 such that we have $n/4$ sets of size 4.
4. Repeat this process until we have a single set containing n elements.

Here is a particular instance of this scheme,

```
WeightedUnion(x_1, x_2)
WeightedUnion(x_3, x_4)
WeightedUnion(x_5, x_6)
WeightedUnion(x_7, x_8)
...
WeightedUnion(x_1, x_3)
WeightedUnion(x_5, x_7)
WeightedUnion(x_9, x_11)
...
WeightedUnion(x_1, x_5)
WeightedUnion(x_9, x_13)
...
WeightedUnion(x_1, x_{n/2})
```

1. The running time for creating $n/2$ disjoint sets of size 2 is $cn/2$ since x_i.`head` is updated for $n/2$ elements and each update takes constant time.

2. The running time for creating $n/4$ disjoint sets of size 4 is $cn/2$ since x_i.`head` is updated for $2 \cdot (n/4) = n/2$ elements because each set has 2 elements and $n/4$ of the sets will be updated.

3. The running time for creating $n/8$ disjoint sets of size 8 is $cn/2$ since x_i.`head` is updated for $4 \cdot (n/8) = n/2$ elements because each set has 4 elements and $n/8$ of the sets will be updated.

4. In general, the running time for creating $n/2^k$ disjoint sets of size 2^k from $n/2^{k-1}$ sets of size 2^{k-1} is $cn/2$ since x_i.`head` is updated for $2^{k-1} \cdot (n/2^k) = n/2$ elements because each set has 2^{k-1} elements and $n/2^k$ of the sets will be updated.

From this we can see that there will $\log_2(n)$ steps in this procedure and each step takes $n/2$ time, therefore the Amortized running time is bounded below by

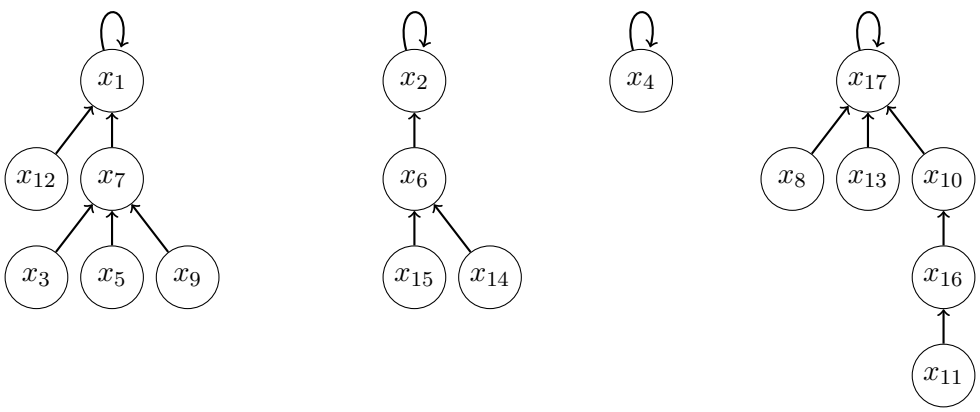
$$T(n) \geq \frac{cn}{2} \log_2(n)$$

Thus, the Amortized time of n `WeightedUnion` operations is in $\Theta(\log_2(n))$.

Tree based Union-Find

Our second implementation of the Union-Find abstract data type will be tree-based Union-Find where each disjoint set is represented as a tree. It is worth mentioning that a tree is a generalization of a linked list, so we should expect this implementation to be at least as efficient as the linked list implementation. Below we can see four trees that correspond to the sets

$$S_1 = \{x_1, x_{12}, x_7, x_3, x_5, x_0\}$$
$$S_2 = \{x_2, x_6, x_{15}, x_{14}\}$$
$$S_3 = \{x_4\}$$
$$S_4 = \{x_{17}, x_8, x_{13}, x_{10}, x_{16}, x_{11}\}$$



The set representative of each tree will the root of the tree and we distinguish the root from other nodes by making the parent of the root itself.

Now, we shall discuss how to implement all of the required methods,

```
function MakeSet(x)
    x.parent = x
end function
```

This method clearly takes $\Theta(1)$ time.

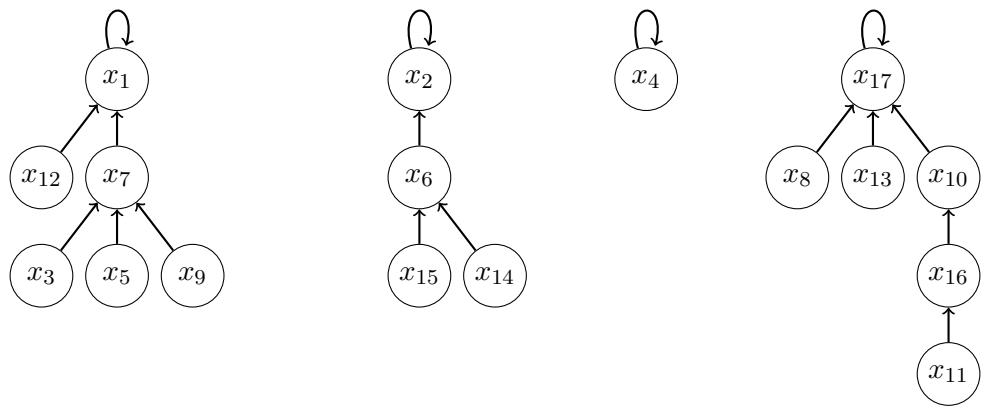
```
function FindSet(x)
    if x.parent = x then
        return x
    else
        z = FindSet(x.parent)
        return z
    end if
end function
```

In order to find the set representative we need to ascend the tree until we find the parent of the node. The running time of this method is $O(z.\texttt{height})$, but we know nothing about this qunatity.

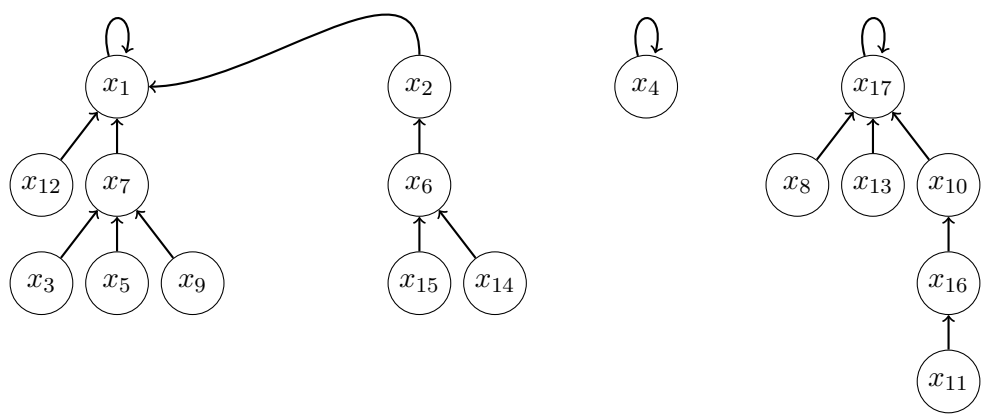
To union two sets together we first find the set representatives and then make the root of the second tree a direct descendent of the root of the first tree.

```
function Union(x,y)
    x' = FindSet(x)
    y' = FindSet(y)
    y'.parent = x'
end function
```

The running time of the method is clearly $O(\max\{x'.\texttt{height}, y'.\texttt{height}\})$ due to the running time of both of the `FindSet` operations. To demonstrate how the method works, consider the trees from before



The result of $\texttt{Union}(x_7, x_{15})$ would be



where the algorithm identifies x_1 and x_2 as the roots of the trees and modifies x_2 to be the child of x_1 since it is the second argument. The reader may have already realized that we can make this method more efficient by doing a weighted union. In particular we shall always perform Unions to minimize the height of the tree since the running time of all of our methods depends upon the height. Therefore, we introduce the modification to the union method,

```
function UnionByHeight(x,y)
    x' = FindSet(x)
    y' = FindSet(y)
    if x' = y' then
        return
    end if
    if x'.height > y'.height then
        y'.parent = x'
    else if y'.height > x'.height then
        x'.parent = y'
    else
        y'.parent = x'
        x'.height = x'.height + 1
    end if
end function
```

In this modified implementation we always add make the shorter tree (i.e. that tree whose height is smaller) the child of the taller tree. In the event that the trees have the same height we shall resort to our old implementation; in this instance the height of the tree is increased by one.

The question now becomes: can we determine anything about the heights of the trees corresponding to the disjoint sets? Namely, we wish to determine the height of each tree in terms of the total number of elements in the disjoint sets. We begin by first proving that the x.`height` field stored at the root of the tree is in fact the height of tree. In the past we have taken for granted that various fields are the same as their mathematical counterpart, but without proving it rigorously, we cannot use that fact with certainty in other proofs. The reader may recall that we had to perform a similar analysis when discussing the Shortest Path Tree when we proved that the distance field was in fact the shortest path distance between the nodes; the proof we shall present here is the same idea, but the code is noticeably simpler for Union-Find.

Proposition 1. If r is the root of tree T, then $r.\texttt{height}$ is the height of the tree.

Proof. We prove it by induction on the number of vertices.
Base case: If a tree T has one vertex, r, then $r.\texttt{height}$ is 0 and height(T) is 0.
Inductive hypothesis: If tree T has fewer than n and r is the root of tree T, then $r.\texttt{height} =$ height(T).
Goal: We want to show that if tree T has $n > 1$ vertices and r is the root of T, then $r.\texttt{height} =$ height(T).
Let tree T be created by $\texttt{UnionByHeight}(x, y)$ where x is in tree T_x and y is tree T_y. Let x' be the root of T_x and y' be the root of T_y. By inductive assumption $x'.\texttt{height} =$ height(T_x) and $y'.\texttt{height} =$ height(T_y).

Case 1 $x'.\texttt{height} > y'.\texttt{height}$.
According to the code, $r.\texttt{height} = x'.\texttt{height}$; by inductive assumption $x'.\texttt{height} =$ height(T_x). Since $x'.\texttt{height} > y'.\texttt{height}$, the length of the longest path from r to nodes in T_y is no longer than the length of the longest path from r to nodes in T_x; thus height$(T_x) =$ height(r)

Case 2 $x'.\texttt{height} < y'.\texttt{height}$.
Same argument as **Case 1**.

Case 3 $x'.\texttt{height} = y'.\texttt{height}$.
$r.\texttt{height} = x'.\texttt{height} + 1 =$ height$(T_x) + 1 =$ height(r); the leaf that is farthest from the root will be a node in y and the length of the path will be one more than the height of T_y.

□

With this result in hand we can now begin attempting to determining a relationship between the number of nodes in a given tree and the height.

In the following proof size(T) is the number of nodes in tree T.

Proposition 2. If T has height h, then size$(T) \geq 2^h$

Proof. We prove it by induction on the size(T).
Base case: If size$(T) = 1$, then height$(T) = 0$ and size$(T) = 1 \geq 2^{\text{height}(T)}$
Inductive assumption: If size$(T) < n$, then size$(T) \geq 2^{\text{height}(T)}$.
Goal: We want to show that if size$(T) = n > 1$, then size$(T) \geq 2^{\text{height}(T)}$.
Suppose tree T was created by `UnionByHeight`(x, y) where x is in the tree T_x and y is in tree T_y. Let x' be the root of tree T_x and y' be the root of tree T_y. By inductive assumption size$(T_x) \geq 2^{\text{height}(T_x)}$ and size$(T_y) \geq 2^{\text{height}(T_y)}$.

Case 1 x'.`height` $>$ y'.`height`.
By Proposition 1, `height`$(T) =$ `height`(T_x). Furthermore,

$$\text{size}(T) = \text{size}(T_x) + \text{size}(T_y) \geq \underbrace{\text{size}(T_x) \geq 2^{\text{height}(T_x)}}_{\text{by inductive assumption}} = 2^{\text{height}(T)}$$

Case 2 x'.`height` $<$ y'.`height`.
Same argument as **Case 1**.

Case 3 x'.`height` $=$ y'.`height`.
By Proposition 1, height$(T) =$ height$(T_x) + 1$ and height$(T_x) =$ height(T_y). Now,

$$\text{size}(T) = \text{size}(T_x) + \text{size}(T_y) \geq 2^{\text{height}(T_x)} + 2^{\text{height}(T_y)} = 2 \cdot 2^{\text{height}(T_x)} = 2^{\text{height}(T_x)+1}$$

and height$(T) =$ height$(T_x) + 1$, so

$$\text{size}(T) \geq 2^{\text{height}(T)}$$

□

Just as we saw when analyzing binary search trees, a lower bound on the number of nodes in a tree (in terms of the height) can be used to determine an upper bound on the height. Therefore, our final result achieves this bound.

Corollary 1. If T has height h, then $h \leq \log_2(\text{size}(T))$.

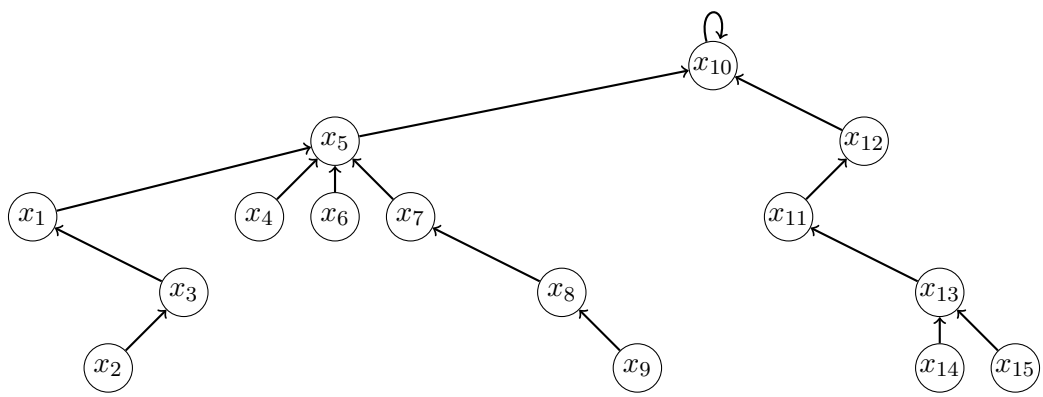
Proof. By Proposition 2, size$(T) \geq 2^h$ so $h \leq \log_2(\text{size}(T))$ □

It is worth noting that a similar analysis cannot necessarily be done to achieve a matching lower bound. Having achieved this upper bound, we again ask ourselves: can we do any better? If the reader thinks about this problem for a little while they may realize that we can "compress" the path when we find the set representative of a node. That is, any time that we call the `FindSet` function, why not make

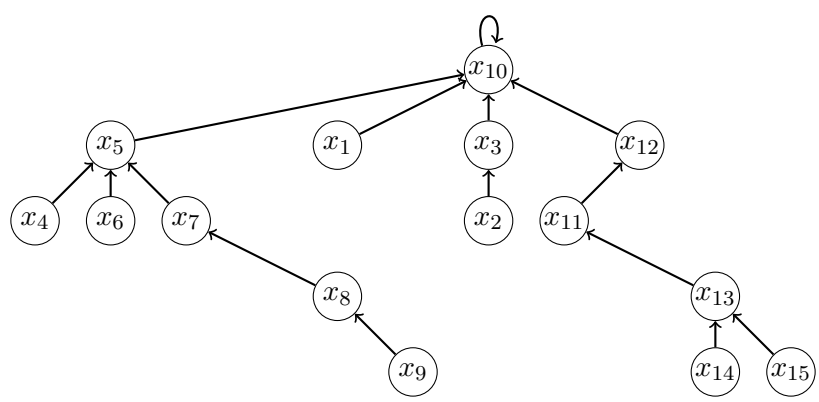
every node along the path from the argument to the root point at the root as the parent? Doing so will hopefully reduce the overall height of the trees.; this technique is known as path compression.

```
function FindSetPathCompress(x)
    if x.parent = x then
        return x
    else
        z = FindSetPathCompress(x.parent)
        x.parent = z
        return z
    end if
end function
```

To demonstrate how this method works, we shall introduce a new tree



Suppose that we perform the operation FindSetPathCompress(x_3), then we take each node along the path from x_3 to the root (x_{10}) and make their parent the root. Doing so the tree becomes



Notice that this operation did not actually reduce the height of the tree. This (among other reasons) makes analyzing this method well beyond the scope of this course. That being said, we shall at least comment on the running time of this method for any reader that is interested.

Firstly, we shall replace the UnionByHeight with UnionByRank that will perform the exact same type of operation, but shall perform unions to minimize the rank of each tree, where height$(x) \leq x$.rank. In

this case, the running time of m union operations on disjoint sets containing n elements is $O(m\alpha(n))$, where α is the inverse Ackerman function. For those unaware, the Ackerman function is a famously "fast growing" function The Ackerman function is defined recursive by the following:

$$A(m,n) = \begin{cases} n+1 & m = 0 \\ A(m-1,1) & m = 0 \\ A(m-1, A(m,n-1)) & \text{otherwise} \end{cases}$$

To claim that this function grows quickly is a sever understatement; for example $A(4,2)$ is an integer with $19,729$ digits. Since the Ackerman function grows so quickly, the inverse therefore grows proportionally slowly; namely $\alpha(n) \leq 5$ for any value of $n \leq A(4,4) \leq A(4,2) = 2^{65536} - 3$. Which means that, for all purposes in our universe, the amortized running time of Union By Rank will be constant.